

Examen Parcial: Estructuras de Datos en el Universo Pokémon

Instrucciones Generales

- Este examen está diseñado para evaluar su comprensión y aplicación de los conceptos de estructuras de datos, incluyendo TADs, listas enlazadas, pilas, filas, árboles, árboles binarios y árboles binarios de búsqueda.
- Responda cada ejercicio con claridad y justifique cuando sea indicado.
- El examen se basa en el universo Pokémon, utilice su creatividad para implementar lo pedido.

Ejercicio 1: Implementación de TADs

Enunciado

En este universo, cada entrenador tiene un equipo Pokémon. Para este ejercicio deberá implementar dos clases en Python: **Pokemon** y **Entrenador**.

Clases a Implementar

1. Clase **Pokemon**:

◦ **Atributos:**

- **nombre**: (*str*) Nombre del Pokémon.
- **tipo**: (*str*) Tipo del Pokémon (por ejemplo, Agua, Fuego, Planta).
- **nivel**: (*int*) Nivel del Pokémon (debe estar entre 1 y 100).

◦ **Métodos:**

- **__init__()**: Constructor que inicializa los atributos del Pokémon.
- **subir_nivel()**: Método que incrementa el nivel del Pokémon en 1, siempre y cuando no supere el nivel 100.
- **__str__()**: Método que devuelve una representación en cadena del Pokémon en el formato:

2. Clase **Entrenador**:

◦ **Atributos:**

- **nombre**: (*str*) Nombre del entrenador.
- **equipo**: (*lista*) Lista que contiene instancias de la clase **Pokemon**.

◦ **Métodos:**

- **__init__()**: Constructor que inicializa el nombre del entrenador y crea una lista vacía para el equipo.
- **agregar_pokemon()**: Método que agrega un Pokémon al equipo. Si ya hay 6 Pokémon en el equipo, debe mostrar un mensaje indicando que no se pueden tener más Pokémon.
- **mostrar_equipo()**: Método que imprime todos los Pokémon del equipo utilizando el método **__str__** de la clase **Pokemon**.

- `nivel_promedio()`: Método que calcula y devuelve el nivel promedio de los Pokémon en el equipo. Si no hay Pokémon en el equipo, debe devolver 0.

Ejercicio 2: Recursividad en el Universo Pokémon

Enunciado

Los entrenadores a menudo se enfrentan a desafíos que requieren la búsqueda de Pokémones en un área. Para este ejercicio implementar una función recursiva que simula la búsqueda de un Pokémon específico en un árbol de Pokémones.

Contexto

Suponga que tiene un árbol binario donde cada nodo representa un Pokémon, cada nodo tiene un nombre y puede tener hasta dos hijos (uno a la izquierda y otro a la derecha). La búsqueda de un Pokémon específico se puede realizar de manera recursiva verificando si el Pokémon se encuentra en el nodo actual o en uno de sus subárboles.

Clases a Implementar

1. Clase `PokemonNode`:

- **Atributos:**

- `nombre`: (*str*) Nombre del Pokémon.
- `izquierda`: (*PokemonNode*) Hijo izquierdo (puede ser `None`).
- `derecha`: (*PokemonNode*) Hijo derecho (puede ser `None`).

- **Métodos:**

- `__init__(self, nombre: str)`: Constructor que inicializa el nombre del Pokémon y establece los hijos izquierdo y derecho como `None`.

2. Función Recursiva `buscar_pokemon`:

- **Parámetros:**

- `nodo`: (*PokemonNode*) Nodo actual del árbol donde se realiza la búsqueda.
- `nombre_pokemon`: (*str*) Nombre del Pokémon que se busca.

- **Retorno:**

- Devuelve `True` si el Pokémon es encontrado en el árbol, y `False` si no se encuentra.

- **Lógica:**

- Si el nodo actual es `None`, devuelve `False`.
- Si el nombre del nodo actual coincide con `nombre_pokemon`, devuelve `True`.
- Llama recursivamente a sí misma para buscar en el hijo izquierdo y luego en el hijo derecho.

Ejercicio 3: Sistema de Evoluciones de Pokémon

Enunciado

Cada Pokémon podría evolucionar a una forma más poderosa, para gestionarlas implementar un sistema utilizando una **pila** (stack), que permita llevar un registro de las evoluciones en orden inverso ya que la más reciente es la que se aplicará primero.

Contexto

La pila se utilizará para almacenar las evoluciones de un Pokémon. Cuando uno de ellos evoluciona, se agrega su nueva forma a la pila. Si el entrenador decide revertir la evolución se puede desapilar la forma más reciente.

Clases a Implementar

1. Clase **Evolucion**:

- **Atributos:**
 - **nombre**: (*str*) Nombre del Pokémon en su forma evolucionada.
- **Métodos:**
 - **__init__(self, nombre: str)**: Constructor que inicializa el nombre de la evolución.

2. Clase **PilaEvoluciones**:

- **Atributos:**
 - **evoluciones**: (*lista*) Lista enlazada que almacena las evoluciones en orden.
- **Métodos:**
 - **__init__(self)**: Constructor que inicializa la lista de evoluciones como vacía.
 - **apilar(self, evolucion: Evolucion)**: Método que agrega una evolución a la pila.
 - **desapilar(self)**: Método que elimina y devuelve la última evolución agregada a la pila. Si la pila está vacía, debe devolver un mensaje indicando que no hay evoluciones para desapilar.
 - **mostrar_evoluciones(self)**: Método que imprime todas las evoluciones en el orden en que fueron apiladas.

Ejercicio 4: Diseño Teórico de un Juego de Rol de Pokémon

Enunciado

Imagine que debe desarrollar un juego de rol basado en el universo Pokémon. Este juego incluirá elementos como la captura de Pokémon, batallas, gimnasios, insignias y la interferencia del equipo Rocket. Para cada uno de estos componentes deberá explicar, qué estructuras de datos utilizaría y por qué; considerando la eficiencia y eficacia en cada caso.

Componentes mínimos del Juego a considerar

1. **Captura de Pokémon**
2. **Batallas**

3. **Gimnasios y Batallas por Insignias**
4. **Recorrido del Mapa**
5. **Interrupciones del Equipo Rocket**

Consideraciones Finales

Al diseñar este juego de rol es fundamental, considerar no solo, la estructura de datos más adecuada para cada componente sino también cómo estas estructuras interactúan entre sí. La elección de estructuras eficientes además de mejorar el rendimiento del juego también proporciona una mejor experiencia al jugador al permitir interacciones fluidas y rápidas.

Criterios de Evaluación

1. Corrección y Complejidad del Código (30 puntos)

- Implementación correcta de las clases y métodos solicitados.
- Uso adecuado de las estructuras de datos.
- Complejidad algorítmica y eficiencia del código.

2. Claridad y Legibilidad (20 puntos)

- Estructura y organización del código.
- Uso de comentarios que expliquen la lógica y el propósito del código.
- Nombres de variables y métodos descriptivos.

3. Justificación Teórica (40 puntos)

- Explicación clara y lógica de las elecciones de estructuras de datos en el ejercicio teórico.
- Consideración de la eficiencia y eficacia en el uso de las estructuras seleccionadas.
- Integración creativa y coherente de los elementos del universo Pokémon en el diseño del juego.

4. Manejo de Casos Especiales (10 puntos)

- Capacidad para manejar errores y condiciones especiales en la implementación.
- Robustez del código frente a entradas inesperadas o situaciones límite.

Puntaje Total Universo Pokemon: 100 pikachus

Dato

Los estudiantes disponen de **4 horas** para resolver el examen. Se recomienda gestionar bien el tiempo para abordar cada ejercicio con la profundidad necesaria.

¡Buena suerte!